Data Structures

Topic

Recursive Problem Solving in C++

Objectives

- To understand recursion as a problem-solving technique.
- To apply recursion in mathematical, string, and structural problems.
- To develop logical thinking through base cases and recursive cases.
- To explore recursion in real-life inspired scenarios (staircase, maze, linked list).

Outcomes

By completing these tasks, students will be able to:

1. Implement recursion in mathematical problems (Factorial, LCM).
2. Manipulate and analyze strings recursively (Reverse, Palindrome).
3. Solve combinatorial problems using recursion (Staircase climbing).
4. Apply recursion in backtracking (Rat in a Maze).
5. Traverse and process linked structures recursively (Linked List printing).

Content

- **Mathematical Recursion**: Factorial, GCD & LCM.
- **String Recursion**: Reversing a string, Palindrome checking.
- **Combinatorial Recursion**: Staircase climbing ways.
- **Backtracking with Recursion**: Rat in a Maze path finding.
- **Recursive Data Structure Traversal**: Printing linked list nodes.

Task 1

Write a recursive function, power, that takes as parameters two integers x and y such that x is nonzero and returns x^y. You can use the following recursive definition to calculate x^y. Consider all cases if y is >0 and y<0 and y==0.

Task 2

Reversing a string is a common operation in text processing, and recursion offers an elegant way to solve it. Your task is to write a recursive function in C++ that reverses a given string without using loops or built-in reverse functions. The function should swap the first and last character of the string, then recursively reverse the remaining substring. For example, the input "HELLO" should produce the output "OLLEH". This task will help you understand how recursion can be used to manipulate strings by focusing on base cases and combining results from smaller subproblems.

Task 3

A palindrome is a word, phrase, or number that reads the same forwards and backwards, such as "MADAM" or "LEVEL". Your task is to write a recursive function in C++ to check whether a given string is a palindrome. The function should compare the first and last characters, and if they match, recursively check the substring in between. If a mismatch is found, the function should immediately return false. For example, the input "MADAM" should return "Palindrome", whereas the input "HELLO" should return "Not a Palindrome". This task emphasizes how recursion is naturally suited for problems involving repeated comparisons from both ends toward the center.

Task 4

The least common multiple (LCM) of two numbers is the smallest number that is a multiple of both. Write and test a method LCM with the following specification.
PARAMETERS: positive integers j and k
RETURN VALUE: the least common multiple (LCM) of j and k

EXAMPLES: LCM (3, 5) is 15
LCM (6, 8) is 24

Task 5

In real life, consider a staircase with n steps. A person standing at the bottom can climb the staircase by taking either 1 step or 2 steps at a time. For example, if the staircase has 3 steps, the person can reach the top in 3 different ways: (1+1+1), (1+2), or (2+1). Your task is to write a recursive function in C++ that calculates the total number of distinct ways to climb to the top of a staircase of size n. The recursive relation can be defined as:

- If there are 0 steps, there is only 1 way (doing nothing).
- If there is 1 step, there is only 1 way (a single step).
- For any n > 1, the number of ways to climb n steps is the sum of the ways to climb n-1 steps and n-2 steps, because the last move could have been either a single step or a double step.

This recursive problem is a classic example of how complex scenarios can be broken down into smaller subproblems. It also builds a strong foundation for understanding dynamic programming concepts, as the recursive approach generates overlapping subproblems that can be optimized later.

ExampleInput:n=4
Example Output: 5 (The ways are: 1+1+1+1, 1+2+1, 2+1+1, 1+1+2, 2+2)

Task 6

In linked lists, traversal is often done using loops, but recursion can also be used effectively to visit each node. Your task is to implement a recursive function in C++ that prints all elements of a singly linked list using only the head pointer. The idea is simple: if the list is empty (base case), stop. Otherwise, print the current node's data and recursively call the function on the next node. This problem strengthens understanding of recursion with linked structures, as each recursive call reduces the problem size by moving one step ahead in the list.

Task 7

Write a recursive function that takes an integer as input and returns its reverse. For example, if the input number is `1234`, the function should return `4321`.

Task 8

Write a recursive function that converts a decimal number into its binary equivalent. The function should display or return the binary representation of the given number.

Task 9

Imagine a rat placed at the starting point (0,0) of a maze represented as a grid (matrix). The maze contains open paths (1) and blocked paths (0). The rat can only move **down** or **right** at each step. The goal is to find a path for the rat to reach the destination (n-1, n-1) using recursion. Your task is to implement a recursive function in C++ that explores possible moves, marking the path taken, and backtracking if a dead end is encountered.

The recursive logic works as follows:

- If the current cell is outside the maze boundaries or blocked, return false.
- If the current cell is the destination, return true.
- Otherwise, move recursively either right or down and check if a path exists.
- If a move doesn't lead to a solution, backtrack by unmarking the current cell and try another path.

This problem not only demonstrates recursion but also introduces the concept of **backtracking**, where the algorithm explores possible solutions and undoes steps when they lead to failure.

Example Input Maze (4×4):

1 0 0 0

1 1 0 1

0 1 0 0

1 1 1 1